

TCP & UDP

- Topics covered:
 - Basics of Layer 4
 - TCP (Transmission Control Protocol)
 - UDP (User Datagram Protocol)
 - Comparing TCP & UDP

Previous Layers Recap

RECAP:

- Layer 1
 - Covered common cable types used in networks
- Layer 2
 - Covered MAC addresses
 - Switching
 - Spanning Tree
- Layer 3
 - Covered IP addresses
 - Routing

Now is a good time to move to Layer 4.

Basics of Layer 4

Functions of Layer 4 Protocols

1. Transparent Transfer of Data Between End Hosts

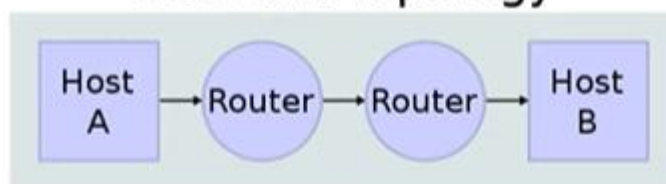
Layer 4 provides transparent transfer of data between end hosts.

This means:

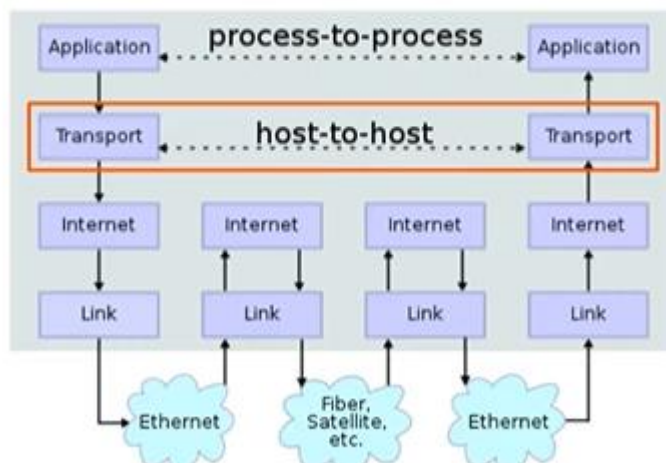
- The Transport Layer encapsulates data with a Layer 4 header
- It then uses Layers 3, 2, and 1 to deliver the data unchanged
- End hosts are not aware of the underlying network details

Network Topology:

Network Topology



Data Flow



en:User:Kbrose (https://commons.wikimedia.org/wiki/File:IP_stack_connections.svg), „IP stack connections“, <https://creativecommons.org/licenses/by-sa/3.0/legalcode>

2. Services Provided (or Not Provided) to Applications

Layer 4 may provide services to applications.

- TCP provides these services
- UDP does not

Services include:

- Reliable data transfer
Ensures all data is received
- Error recovery
Retransmits data if errors occur
- Data sequencing
Reorders data if it arrives out of order
- Flow control
Prevents sending data faster than the destination can handle

These services are provided by TCP, not UDP.

3. Layer 4 Addressing — Port Numbers

Layer 4 uses port numbers.

- Layer 4 ports are not physical ports
- They identify:
 - Application Layer protocols
 - Communication sessions

Sessions and Session Multiplexing

Definition of a Session

A session is:

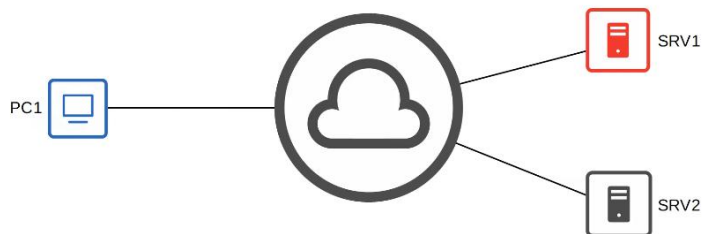
- An exchange of data between two or more devices

A PC must handle:

- Multiple sessions at the same time
- Multiple services
- Multiple tabs or applications

Port Numbers in Action (PC1, SRV1, SRV2)

Initial Topology:

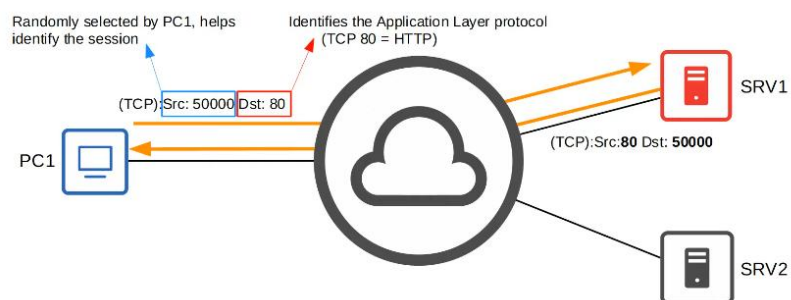


1] PC1 Communicating with SRV1 (HTTP)

- Protocol: TCP
- Source Port: 50000
- Destination Port: 80

Meaning:

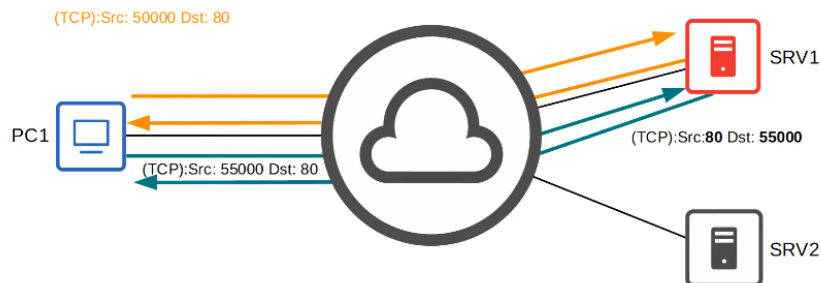
- Destination port 80 identifies HTTP
- Source port 50000 is randomly selected
- Together, they identify the session



Multiple Sessions to the Same Server

2] PC1 opens another connection to SRV1:

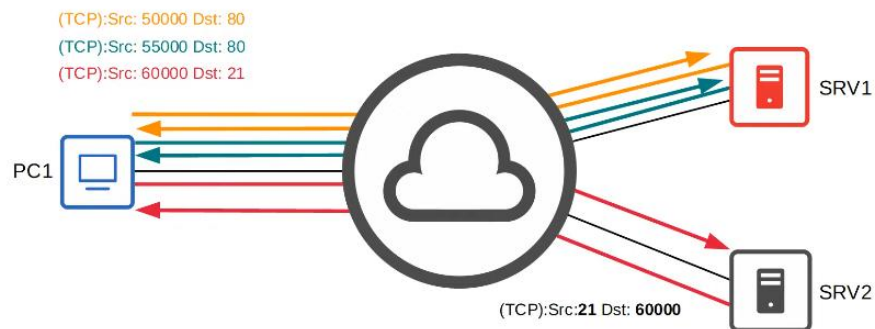
- Destination Port: 80
- Source Port: 55000



PC1 identifies sessions by source + destination ports.

3] PC1 Communicating with SRV2 (FTP)

- Destination Port: 21
- Source Port: 60000
- Protocol: TCP



- Port 21 identifies FTP
- Used for file transfer

Summary of Port Usage

- Ports identify:
 - Application Layer protocol (HTTP, FTP)
 - Individual communication sessions
- Hosts use ports to manage multiple sessions simultaneously

3. Port Number Ranges (IANA)

Port numbers are registered by IANA (Internet Assigned Numbers Authority).

- Well-known ports:
 - Range: 0 – 1023
 - Used for major protocols (HTTP, FTP)
 - Strictly regulated
- Registered ports:
 - Range: 1024 – 49151
 - Registration required (less strict)
- Ephemeral ports:
 - Range: 49152 – 65535
 - Used as random source ports

All previous random source ports came from the ephemeral range.

TCP and UDP

- Port numbers are used by both TCP and UDP
- TCP provides:
 - Reliable data transfer
 - Error recovery
 - Sequencing
 - Flow control
- UDP does not provide these

TCP (Transmission Control Protocol)

Transmission Control Protocol

TCP is a connection-oriented protocol.

Meaning:

- Hosts establish a connection before sending data
- Data exchange starts only after connection setup

TCP Reliability

- Destination host must acknowledge each TCP segment
- If acknowledgment is not received:
 - Segment is retransmitted

Layer 4 PDU name: Segment

TCP Sequencing

- TCP header contains a sequence number
- Allows reordering of segments if they arrive out of order

TCP Flow Control

- Destination host controls the sending rate
- Prevents being overwhelmed by traffic

TCP Header

		TCP segment header																															
Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset				Reserved 0 0 0			N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if data offset > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

Encapsulation View:



Important Fields:

- Source Port (16 bits)
- Destination Port (16 bits)
 - Total ports: $2^{16} = 65536$
- Sequence Number
- Acknowledgment Number
- Flags
 - ACK
 - SYN
 - FIN
- Window Size
 - Used for flow control

No need to memorize entire header, only be aware of key fields.

TCP Three-Way Handshake (Connection Establishment)

Used to establish a connection before data transfer.

Scenario: PC1 accessing SRV1 via HTTP

PC1 ---> SRV1 : SYN

PC1 <--- SRV1 : SYN + ACK

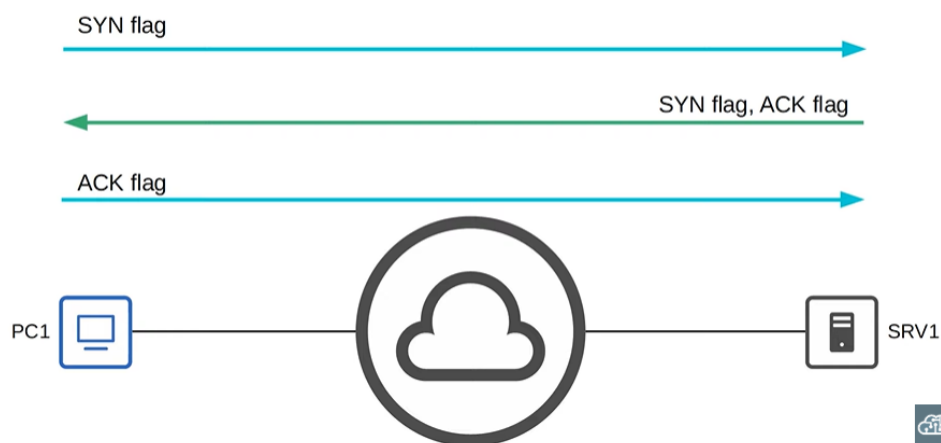
PC1 ---> SRV1 : ACK

Steps

1. SYN — PC1 initiates connection
2. SYN-ACK — SRV1 responds
3. ACK — PC1 confirms

Connection is now established

Data transfer begins after this



TCP Connection Termination (Four-Way Handshake)

Used to terminate a connection.

PC1 ---> SRV1 : FIN

PC1 <--- SRV1 : ACK

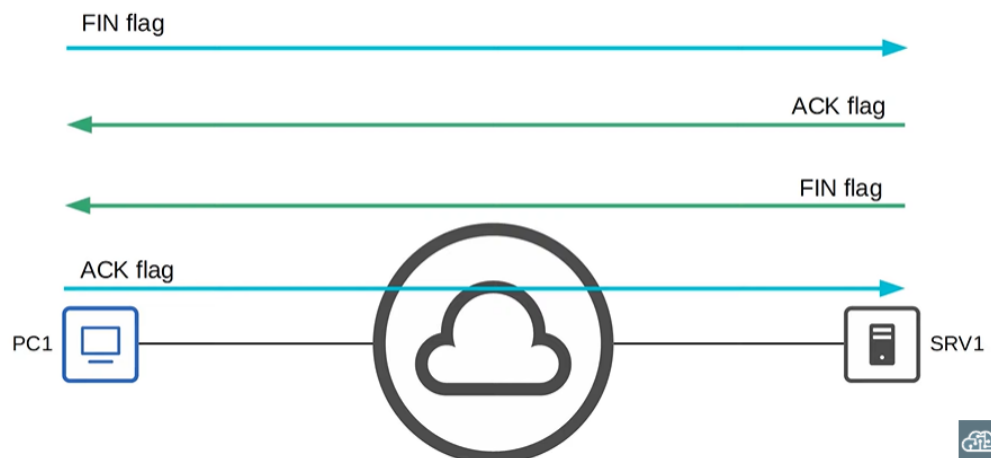
PC1 <--- SRV1 : FIN

PC1 ---> SRV1 : ACK

Steps

1. PC1 sends FIN
2. SRV1 sends ACK
3. SRV1 sends FIN
4. PC1 sends ACK

Connection is terminated



TCP Reliability, Sequencing, Flow Control & UDP

- TCP is connection-oriented.
 - ↳ Before actually sending data to the destination host, the two hosts communicate to establish a connection. Once the connection is established, the data exchange begins.
- TCP provides reliable communication.
 - ↳ The destination host must acknowledge that it received each TCP segment.
 - ↳ If a segment isn't acknowledged, it is sent again.
- TCP provides sequencing.
 - ↳ Sequence numbers in the TCP header allow destination hosts to put segments in the correct order even if they arrive out of order.
- TCP provides flow control.
 - ↳ The destination host can tell the source host to increase/decrease the rate that data is sent.

TCP Sequence and Acknowledgment Numbers

TCP uses the sequence number and acknowledgment number fields of its header to provide reliable communication and sequencing.

To demonstrate this, consider communication between PC1 and PC2.

[PC1] ----- (Internet) ----- [PC2]

Sequence Numbers During the Three-Way Handshake

When PC1 sends the SYN message of the three-way handshake, it sets a random initial sequence number, for example 10.

PC1 ---> PC2 : Seq 10 (SYN)

PC2 replies with a SYN-ACK.

PC2 sets its own random initial sequence number, for example 50, and acknowledges PC1's sequence number by setting the acknowledgment number to 11.

PC1 <--- PC2 : Seq 50 , Ack 11 (SYN-ACK)

This is because TCP uses forward acknowledgment.

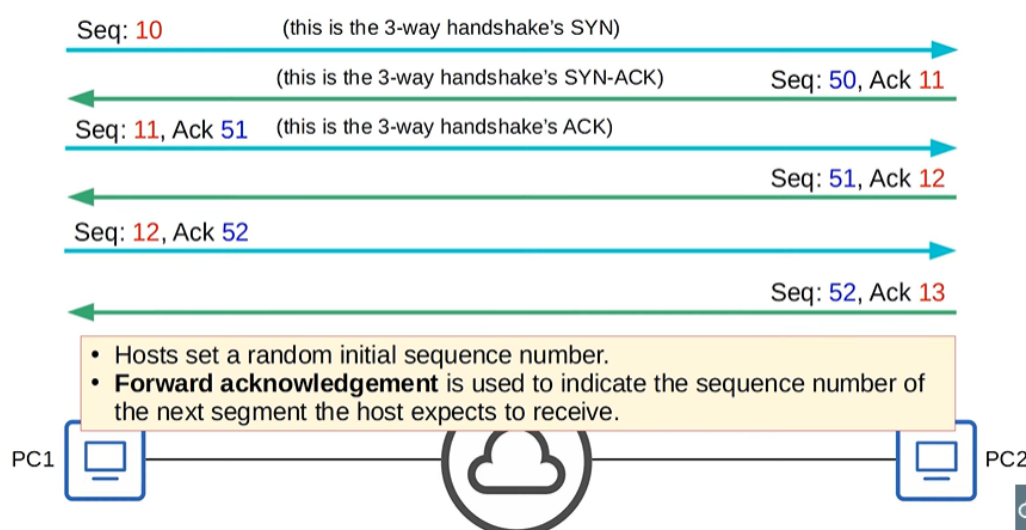
Instead of acknowledging sequence number 10 directly, it indicates the next sequence number it expects to receive.

PC1 then sends the final ACK of the handshake using sequence number 11 and acknowledgment number 51.

PC1 ---> PC2 : Seq 11 , Ack 51 (ACK)

PC2 replies with sequence number 51 and acknowledgment number 12, and the exchange continues.

Representation of the Exchange



What to Remember

- Hosts set random initial sequence numbers
- TCP uses forward acknowledgment to indicate the next expected segment

TCP

- TCP is connection-oriented.
 - ↳ Before actually sending data to the destination host, the two hosts communicate to establish a connection. Once the connection is established, the data exchange begins.
- TCP provides reliable communication.
 - ↳ The destination host must acknowledge that it received each TCP segment.
 - ↳ If a segment isn't acknowledged, it is sent again.
- TCP provides sequencing.
 - ↳ Sequence numbers in the TCP header allow destination hosts to put segments in the correct order even if they arrive out of order.
- TCP provides flow control.
 - ↳ The destination host can tell the source host to increase/decrease the rate that data is sent.

TCP Retransmission (Unacknowledged Segment)

Now consider what happens when a segment is not acknowledged.

To simplify, only one set of sequence and acknowledgment numbers is shown.

PC1 sends a segment to SRV1 with sequence number 20.

PC1 ---> SRV1 : Seq 20

SRV1 replies with Ack 21, using forward acknowledgment.

PC1 <--- SRV1 : Ack 21

PC1 then sends the next segment with sequence number 21, but this segment is lost.

PC1 ---> SRV1 : Seq 21 X (lost)

PC1 waits for an acknowledgment, and after no response, it resends the segment.

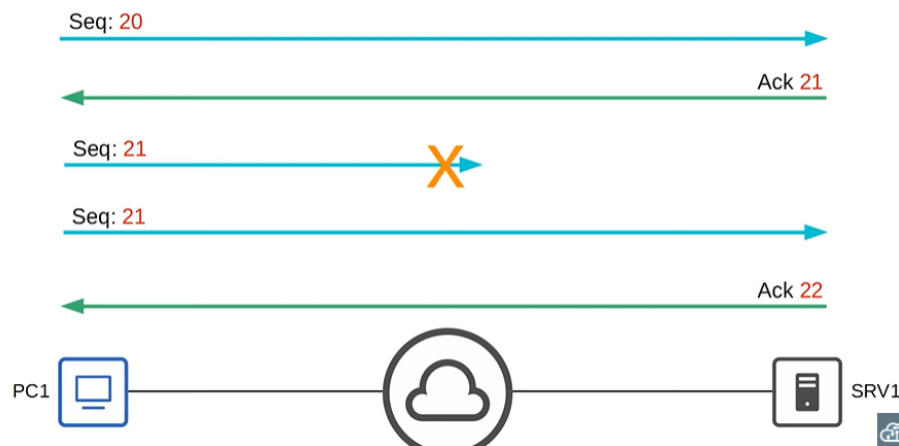
PC1 ---> SRV1 : Seq 21 (retransmission)

This is called TCP retransmission.

SRV1 receives the retransmitted segment and replies with Ack 22.

PC1 <--- SRV1 : Ack 22

Representation



This shows how TCP ensures reliable delivery by retransmitting unacknowledged segments.

TCP

- TCP is connection-oriented.
 - ↳ Before actually sending data to the destination host, the two hosts communicate to establish a connection. Once the connection is established, the data exchange begins.
- TCP provides reliable communication.
 - ↳ The destination host must acknowledge that it received each TCP segment.
 - ↳ If a segment isn't acknowledged, it is sent again.
- TCP provides sequencing.
 - ↳ Sequence numbers in the TCP header allow destination hosts to put segments in the correct order even if they arrive out of order.
- TCP provides flow control.
 - ↳ The destination host can tell the source host to increase/decrease the rate that data is sent.

TCP Flow Control

Acknowledging every individual segment is inefficient.

TCP solves this using the **Window Size** field in the TCP header.

The window size allows multiple segments to be sent before an acknowledgment is required.

For example, a host may send segments with sequence numbers **20, 21, and 22**, and then receive a single acknowledgment with **Ack 23**.

Representation

PC1 ---> : Seq 20

PC1 ---> : Seq 21

PC1 ---> : Seq 22

PC1 <--- : Ack 23

TCP also uses a **sliding window**.

- The window size increases until a segment is dropped
- When a segment is dropped, the window size is reduced
- The window then slowly increases again

In real networks, sequence numbers are much larger and do not increase by 1.

For the CCNA exam, understanding the **concept** is sufficient.

Purpose of This TCP Explanation

This section demonstrated how TCP provides:

- Reliable communication
- Sequencing
- Retransmission
- Flow control

The CCNA exam requires understanding these concepts in order to compare TCP and UDP, not detailed TCP mechanics.

Comparing TCP & UDP

TCP Header vs UDP Header

TCP segment header																																	
Offsets	Octet	0								1								2								3							
Octet	Bit	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
0	0	Source port																Destination port															
4	32	Sequence number																															
8	64	Acknowledgment number (if ACK set)																															
12	96	Data offset		Reserved 0 0 0		N S	C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size																		
16	128	Checksum																Urgent pointer (if URG set)															
20	160	Options (if data offset > 5. Padded at the end with "0" bytes if necessary.)																															
...	...	...																															

UDP datagram header																																		
Offsets	Octet	0								1								2								3								
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
0	0	Source port																Destination port																
4	32	Length																Checksum																



When TCP or UDP Is Used

TCP provides more features but introduces more overhead.
Acknowledgments and retransmissions can slow data transfer.

TCP is preferred for applications that require **reliable communication**, such as file downloads.

UDP is preferred for **real-time applications** like voice and video, where delay is more critical than reliability.

Some applications use UDP but handle reliability themselves, such as **TFTP**.
Some applications use both TCP and UDP, depending on the situation, such as **DNS**.

Layer 4 Port Numbers (Reminder)

Both TCP and UDP:

- Use port numbers
- Provide Layer 4 addressing

- Support session multiplexing

Important Well-Known Port Numbers

TCP

- FTP data (20)
- FTP control (21)
- SSH (22)
- Telnet (23)
- SMTP (25)
- HTTP (80)
- POP3 (110)
- HTTPS (443)

UDP

- DHCP server (67)
- DHCP client (68)
- TFTP (69)
- SNMP agent (161)
- SNMP manager (162)
- Syslog (514)

TCP & UDP

- DNS (53)